

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 1 – Grupo 3

Ejercicio 2: *Modelos de Clasificación Binaria*

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: liencho@fi.uba.ar
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar

Fecha de entrega: 01/05/25

Ejercicio 2: Modelos de Clasificación Binaria

Breve descripción y análisis del dataset

Locaciones asignadas: Queensland / Victoria / Australia Meridional / Australia Occidental

El Dataset contiene información sobre distintas estaciones meteorológicas de Australia. El objetivo es predecir si lloverá o no al día siguiente, volcándolo en la variable RainTomorrow. La cantidad de registros inicial del Dataset (sin filtrar por nuestras locaciones asignadas) fue de 145.460 filas y 23 columnas. Tras la filtración por locations, pasó a ser de 78.014 filas.

Luego del primer análisis, definimos si cada variable era cuantitativa o cualitativa, nominal u ordinal y continua o discreta, respectivamente. A las variables cuantitativas les calculamos sus medidas de resumen y para las cualitativas obtuvimos sus frecuencias. que mostramos mediante tablas y gráficos.

La categorización final fue:

Tipo de variable	Cuantitativas	Tipo de variable	Cualitativas
<u>Continua</u>	MinTemp, MaxTemp, Rainfall, Evaporation, Sunshine, WindGustSpeed, WindSpeed9am, WindSpeed3pm, Pressure9am, Pressure3pm, Temp9am, Temp3pm	<u>Nominal</u>	Date, Location, WindGustDir, WindDir9am, WindDir3pm, RainToday, RainTomorrow
<u>Discreta</u>	Humidity9am, Humidity3pm, Cloud9am, Cloud3pm	<u>Ordinal</u>	—

En cuanto a los datos faltantes, primero analizamos el porcentaje de datos faltantes por columna. Creamos un gráfico y descubrimos que de 23 columnas, 21 de ellas tenían valores faltantes. Antes de imputar los valores faltantes, nos cuestionamos si podíamos eliminar alguna columna pero optamos por no hacerlo porque todas parecen aportar información relevante para lograr el objetivo del trabajo. La imputación de valores faltantes fue realizada con la mediana para las variables cuantitativas y por moda para las cualitativas. De esta forma suprimimos todos los valores NaN en el dataset.

Para visualizar correctamente los valores atípicos, hicimos gráficos boxplot por cada variable cuantitativa y luego creamos una tabla donde mostraba la cantidad de outliers por variable basándose en el rango intercuartílico. Vimos los valores extremos tanto bajos como altos. Todo esto fue para nuestro propio entendimiento del Dataset y sus características, pues para ser más precisos optamos por utilizar IsolationForest para la efectiva filtración de los outliers. Tras la filtración, quedaron 74.113 filas (3901 registros menos).

Entrenamiento y predicción

Un conflicto con el que nos encontramos al momento de ver el rendimiento de los distintos modelos fue que las métricas daban valores muy bajos de Recall. En general, todos los modelos no predecían correctamente RainTomorrow, y eran demasiado buenos prediciendo NotRainTomorrow. Tras mucho indagar, descubrimos que el dataset estaba desbalanceado: un 80% del mismo era de la clase NotRainTomorrow, mientras que un 20% era de la clase RainTomorrow.

```
Visualizándolo con porcentajes:  
RainTomorrow  
0    80.059032  
1    19.940968  
Name: proportion, dtype: float64
```

Para solucionar esto, debatimos si aplicar RandomOverSampling o RandomUnderSampling. El OverSampling para balancear duplica ejemplos existentes de la clase minoritaria. Mientras que el UnderSampling reduce la cantidad de instancias de la clase mayoritaria, eliminando datos.

Optamos por aplicar OverSampling para preservar la información, dado que el tamaño de nuestro dataset era reducido y no queríamos perder información. Evaluamos su conveniencia de manera específica para cada modelo.

Modelo 1: Árbol de decisión

Con el objetivo de construir un Árbol de Decisión, utilizamos la métrica RandomSearchCV y luego hicimos **K-fold Cross Validation** con **5 folds**. Los hiperparámetros que optimizamos fueron:

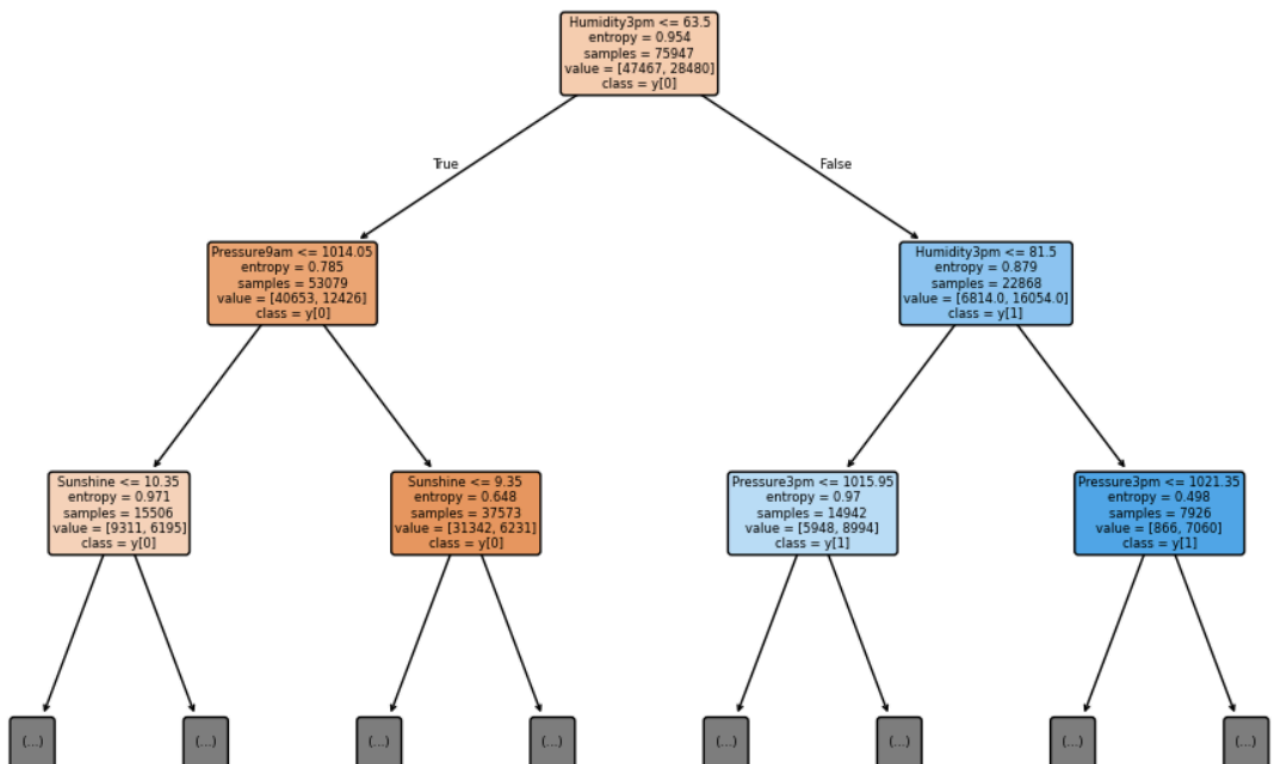
- criterion: criterio de medida para las divisiones ("entropy").
- max_depth: controla la profundidad máxima que puede tener el árbol (randint(4, 10)).

- `min_samples_leaf`: indica la cantidad mínima de muestras que debe tener una hoja de un árbol (`randint(80, 150)`).
- `min_samples_split`: define el número mínimo de muestras necesarias para dividir un nodo (`randint(20, 80)`).
- `ccp_alpha`: controla la poda del árbol y elegimos `uniform(0.000005, 0.00005)` que toma un valor aleatorio dentro de ese rango.

Respecto al OverSampling, lo aplicamos con una `sampling_strategy` de 0.6. El dataset quedó de la siguiente manera:

```
Distribución tras oversampling en train:
RainTomorrow
0    62.500165
1    37.499835
Name: proportion, dtype: float64
```

Dado que el árbol es muy extenso como para mostrarlo completo en el informe, mostraremos hasta el segundo nivel:



Cada nodo interno representa una decisión basada en una característica (feature). Cada rama representa el resultado de esa decisión (True o False). Y cada hoja representa una predicción final.

El nodo raíz es la primera decisión del árbol y en este caso es *Humidity3pm* ≤ 63.5 (si la humedad a las 15:00 hs es menor o igual a 63.5 se toma la rama izquierda, sino la derecha). Este nodo contiene 74.947 muestras y la clase mayoritaria es **NotRainTomorrow**.

En la rama izquierda a *Humidity3pm* ≤ 63.5 , se evalúa si *Pressure9am* ≤ 1014.05 . Esto quiere decir que evalúa si la presión a las 9:00 hs es menor a 1014.05. Tiene 53.079 muestras y su mayoría son **NotRainTomorrow**. La Subrama Izquierda ve si se cumple *Sunshine* ≤ 10.35 que representa la cantidad de horas de sol con 15.506 muestras y en su mayoría **NotRainTomorrow**. Por otro lado, la Subrama Derecha evalúa *Sunshine* ≤ 9.35 y su cantidad de muestras es 37.573 y en su mayoría son **NotRainTomorrow**.

La conclusión a la que llegamos respecto a la rama izquierda y sus subramas es que si hay baja humedad por la tarde (15:00 hs) y si la presión por la mañana (9:00 hs) es menor o igual a 1014.05, se considera la cantidad de horas de sol. En la subrama izquierda, con menos de 10.35 horas de sol, y en la subrama derecha, con menos de 9.35 horas, ambas condiciones se asocian a que no haya lluvia al día siguiente.

Esto significa que, incluso cuando hay menos sol, la baja humedad es el factor determinante que mantiene baja la probabilidad de lluvia.

A su vez en la rama derecha se evalúa si *Humidity3pm* ≤ 81.5 . En este caso tiene 22.868 muestras y en su mayoría **RainTomorrow**. En la Subrama Izquierda se observa que se evalúa *Pressure3pm* ≤ 1015.95 con 14.942 muestras y es **RainTomorrow**. En cambio, la Subrama Derecha observa *Pressure3pm* ≤ 1021.35 . Tiene 7.962 muestras y son **RainTomorrow**.

Observando la rama derecha y sus subramas concluimos que si la humedad a las 15:00hs es menor o igual a 81.5 y si la presión a esa misma hora es menor o igual a 1015.95 es probable que llueva. Esto nos sugiere que una presión relativamente baja junto con alta humedad favorece condiciones de lluvia.

Por otro lado, si la humedad es mayor a 81.5 en la tarde y si la presión es menor o igual a 1021.35, se observa que igual es probable que llueva. Esto indica que incluso con una presión más alta, la presencia de alta humedad es suficientemente significativa para sostener la predicción de lluvia y presión a la tarde fomenta eso.

Modelo 2: Random Forest

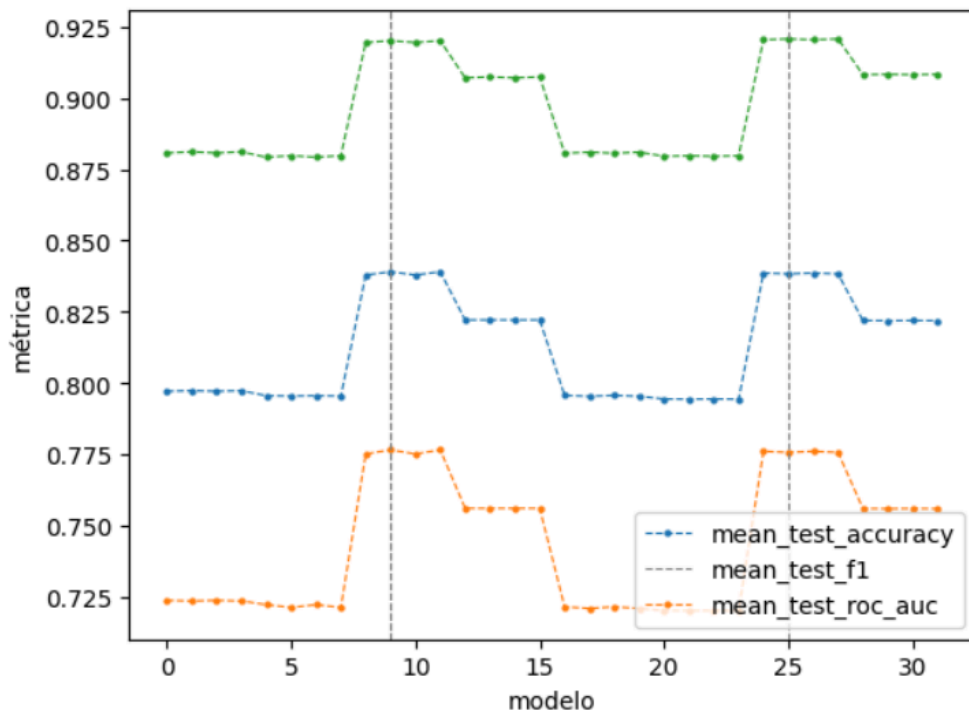
Para crear el Modelo de Random Forest realizamos una **búsqueda de hiperparámetros** utilizando GridSearchCV. Además, utilizamos **K-Fold Cross Validation** con **5 folds** para evaluar el rendimiento de cada combinación de hiperparámetros. Los hiperparámetros optimizados fueron:

- criterion: criterio de medida para las divisiones ("gini" o "entropy").
- min_samples_leaf: cantidad mínima de muestras en una hoja del árbol (10, 20).
- min_samples_split: cantidad mínima de muestras para dividir un nodo (10, 20).
- n_estimators: cantidad de árboles en el bosque (100, 200).
- max_depth: cuán profundo puede crecer cada árbol individual dentro del bosque (8, 116).
- class_weight: que ajuste automáticamente el peso de cada clase en función de cuántas muestras tiene cada una. Así las clases menos representadas tienen más peso, así se les presta más atención. Esto lo que suele mejorar el rendimiento (balanced).

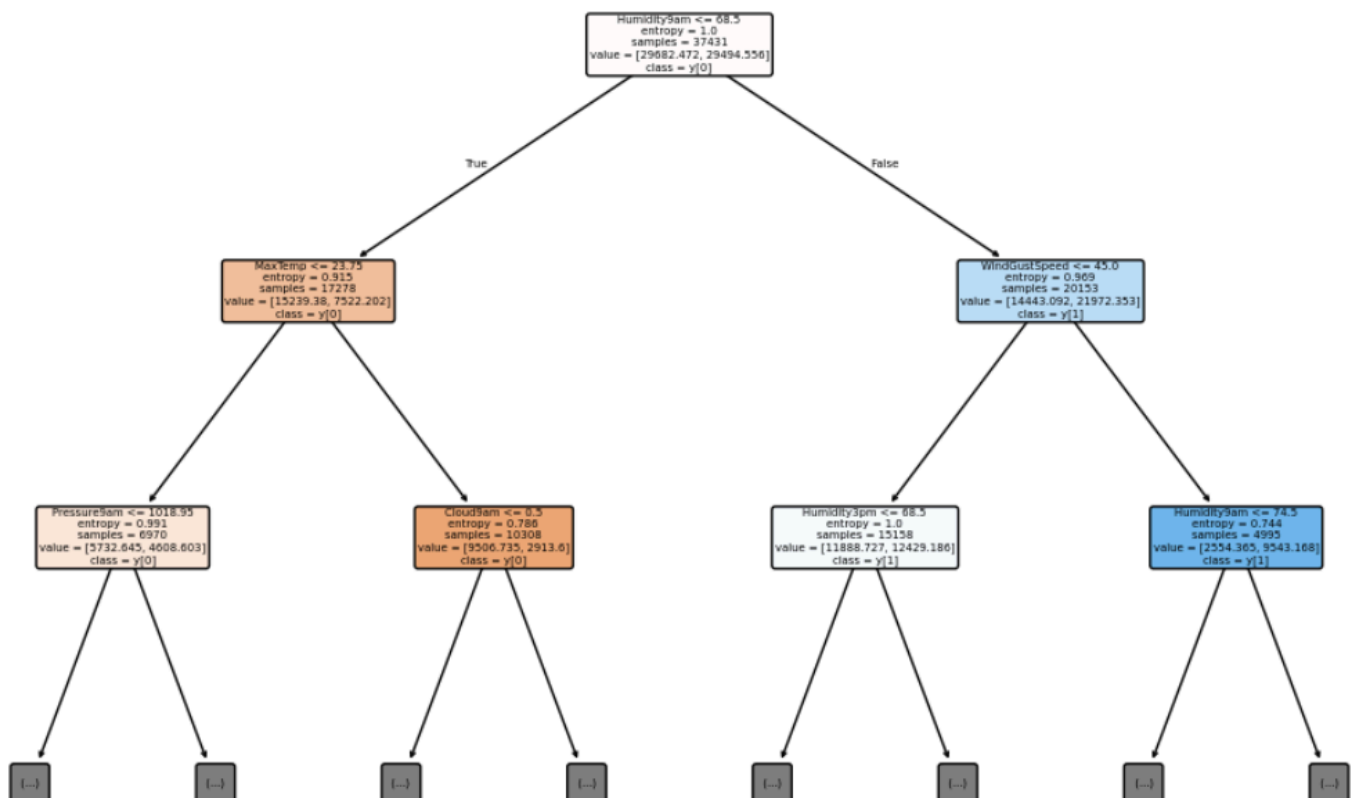
Respecto al OverSampling, lo aplicamos con una sampling_strategy de 0.5. El dataset quedó de la siguiente manera:

```
Distribución tras oversampling en train:  
RainTomorrow  
0    66.667135  
1    33.332865  
Name: proportion, dtype: float64
```

Para elegir la mejor combinación de hiperparámetros seleccionamos **F1 Score** como métrica principal ya que consideramos que permite obtener una performance más representativa del comportamiento real del clasificador, al tener en cuenta tanto la precisión como la sensibilidad del modelo frente a la predicción de días con lluvia. Buscábamos un equilibrio entre precisión y recall.



Como la conformación de los árboles generados es muy extensa, mostramos una porción representativa del árbol 19 para explicar las primeras reglas. Lo graficamos con una profundidad máxima de 2 niveles para facilitar su interpretación:



Cada nodo interno representa una decisión basada en una característica (feature). Cada rama representa el resultado de esa decisión (True o False). Y cada hoja representa una predicción final.

El nodo raíz representa la primera decisión del árbol y en nuestro caso utiliza la variable *Humidity9am* ≤ 70.5 (si la humedad a las 9:00 hs es menor o igual a 70.5 se toma la rama izquierda, si es mayor se toma la rama derecha). Este nodo contiene 45.044 muestras, y la clase mayoritaria es **RainTomorrow**.

En la rama izquierda a *Humidity3pm* ≤ 70.5 , tenemos como primer nivel la variable *MaxTemp* ≤ 20.85 . Evalúa si la temperatura máxima es menor o igual a 20.85°C. Esta división tiene 21.738 muestras, y la mayoría son clasificadas como **NotRainTomorrow**. Ahora pasaremos a describir sus subramas: en la Subrama Izquierda se aplica *Pressure9am* ≤ 1019.45 , que evalúa la presión a las 9:00 hs. Su predicción fue **RainTomorrow** con 6.371 muestras. Por otro lado, en la Subrama Derecha se usa *WindGustSpeed* ≤ 49.0 que evalúa la velocidad de la ráfaga de viento más fuerte en km/h. Su predicción también fue **NotRainTomorrow** (con 15.367 muestras).

La conclusión a la que llegamos respecto a la rama izquierda y sus subramas es que si hay baja humedad en la mañana (9:00 hs) tiende a reducir la probabilidad de lluvia al día siguiente. Además si la temperatura máxima del día es más baja tiende a no llover. Pero también sucede que en condiciones de menor temperatura y humedad, una presión baja favorece la lluvia.

Cuando las ráfagas de viento no son fuertes, y la humedad y temperatura son bajas, es menos probable que llueva. Esta rama y sus subramas muestran cómo la baja humedad mañanera, junto con baja temperatura y ráfagas moderadas hace que haya baja probabilidad de lluvias, aunque una presión mañanera más baja puede revertir un poco dicha tendencia.

En la rama derecha (cuando *Humidity3pm* > 70.5) tenemos como primer nivel a *Rainfall* ≤ 0.45 . Este feature considera la cantidad de lluvia caída (en mm). y agrupa 23.306 muestras. La mayoría de estas se clasifican como **RainTomorrow**. En cuanto a sus subramas: en la izquierda tenemos *WindGustSpeed* ≤ 40.0 con una predicción dominante de **NotRainTomorrow** (habiendo evaluado 13.149 muestras). En el caso de la subrama derecha tenemos *Pressure9am* ≤ 1016.05 , que evalúa la presión a las 9:00 hs y su predicción dominante fue **RainTomorrow** (con un total de 10.157 muestras).

Tras haber observado ambas subramas de la rama derecha, concluimos que cuando hay alta humedad en la mañana y si hay algo de lluvia en el mismo día, es probable que al día siguiente llueva también.

En la subrama izquierda, si hay una baja intensidad del viento parece que puede no llover al día siguiente. En cambio, la subrama derecha evalúa la presión mañanera y muestra que, bajo condiciones húmedas y con presión relativamente baja, se incrementa la probabilidad de lluvias.

Modelo 3: Voting Classifier

Para el modelo a elección, optamos por utilizar un ensamble híbrido, siendo este un **Ensamble por Votación (Voting Classifier)**. El ensamble está compuesto por los siguientes modelos: Regresión Logística, Random Forest y K-Nearest Neighbors (K-NN).

Se optimizaron los siguientes hiperparámetros de cada uno de los clasificadores del ensamble:

- **Regresión Logística (lr)**
 - C: parámetro de regularización (valores continuos entre 0.01 y 1)
 - max_iter: cantidad máxima de iteraciones (valores random 100 y 300)
- **Random Forest (rnd)**
 - n_estimators: cantidad de árboles (valores random entre 30 y 100)
 - max_depth: profundidad máxima del árbol ([3, 5, 10])
 - rnd__min_samples_leaf: cantidad mínima de muestras que debe tener una hoja (valores random entre 5 y 15)
 - rnd__max_features: cuántas variables considerará aleatoriamente para dividir en cada nodo del árbol. Usa la raíz cuadrada del número total de variables. ("sqrt")
- **K-Nearest Neighbors (knn)**
 - n_neighbors: cantidad de vecinos a considerar (valores entre 5 y 10)
 - weights: tipo de ponderación de los vecinos, con uniform todos tienen el mismo peso. (uniform)

Utilizamos la validación cruzada (**K-Cross Validation**) con **4 folds** dentro del procedimiento de búsqueda de hiperparámetros. Esto implica que, para cada combinación de hiperparámetros, el conjunto de entrenamiento fue dividido en 4 partes, utilizando 3 para entrenar y 1 para validar, rotando en cada iteración.

Respecto al OverSampling, lo aplicamos con una sampling_strategy de 0.5. El dataset quedó de la siguiente manera:

```
Distribución tras oversampling en train:  
RainTomorrow  
0    66.667135  
1    33.332865  
Name: proportion, dtype: float64
```

Para llevar a cabo esta búsqueda, se utilizó **RandomizedSearchCV**, el cual realiza una búsqueda aleatoria sobre un número fijo de combinaciones de hiperparámetros (en este caso, n_iter = 30)

La métrica utilizada para evaluar cada combinación de hiperparámetros durante el proceso de búsqueda fue el **F1 Score**. Esto significa que la mejor combinación fue aquella que maximiza el equilibrio entre precisión y recall en las predicciones evaluadas mediante validación cruzada.

Cuadro de resultados

Medidas de rendimiento del conjunto TEST:

<u>Modelo</u>	<u>F1-Test</u>	<u>Precision Test</u>	<u>Recall Test</u>	<u>Accuracy Test</u>
Árbol de Decisión	0.6105	0.5599	0.6711	0.8273
RandomForest	0.6404	0.6569	0.6246	0.8585
Personalizado (Voting)	0.6364	0.6422	0.6306	0.8547

Parámetros elegidos del Árbol de Decisión:

```
{'ccp_alpha': np.float64(2.4146343737689492e-05), 'criterion': 'entropy', 'max_depth': 9, 'min_samples_leaf': 82, 'min_samples_split': 20}
```

Parámetros elegidos del RandomForest:

```
{'class_weight': 'balanced', 'criterion': 'gini', 'max_depth': 16, 'min_samples_leaf': 10, 'min_samples_split': 10, 'n_estimators': 200}
```

Parámetros elegidos del personalizado:

```
{'knn__n_neighbors': 7, 'knn__weights': 'uniform', 'lr__C': np.float64(0.6218528947223795), 'lr__max_iter': 269, 'rnd__max_depth': 10, 'rnd__max_features': 'sqrt', 'rnd__min_samples_leaf': 7, 'rnd__n_estimators': 84}
```

Significado de las distintas medidas de rendimiento:

- **F1-Test:** es la media entre la *precisión* y el *recall*, que busca equilibrar ambos aspectos. Se calcula como $2 * ((\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}))$.
- **Precision Test:** evalúa la proporción de verdaderos positivos sobre el total de predicciones positivas realizadas. Es $TP / (TP + FP)$
- **Recall Test:** mide la capacidad del modelo para identificar correctamente todos los casos positivos. Es $TP / (TP + FN)$.
- **Accuracy Test:** mide la proporción de predicciones correctas sobre el total de predicciones realizadas. Es $(TP + TN) / \text{total de observaciones}$.

Analicemos la performance de todos los modelos respecto al set de entrenamiento.

En el caso del **Árbol de Decisión**, el set de entrenamiento tuvo la siguiente performance:

```
--- Entrenamiento ---  
Accuracy : 0.8087  
Precision: 0.7673  
Recall    : 0.7032  
F1 Score  : 0.7338
```

Vimos que tuvo un rendimiento moderado en el conjunto de test, destacando por un buen recall (0.67), lo que indica que es capaz de detectar bastante bien los días con lluvia. A pesar de eso, su precisión en test fue baja (0.56) comparado con la del entrenamiento (0.76), lo que sugiere varios falsos positivos. Esto implica que el modelo tiende a predecir lluvia incluso cuando no la hay, lo cual puede ser útil si es más importante la lluvia, pero genera muchas falsas alarmas. A pesar de esto, no vimos un gran sobreajuste, ya que el rendimiento en entrenamiento no es excesivamente superior al de test.

Respecto al **RandomForest**, el set de entrenamiento tuvo la siguiente performance:

```
--- Entrenamiento ---  
Accuracy : 0.8995  
Precision: 0.8853  
Recall    : 0.8026  
F1 Score  : 0.8419
```

Consideramos que las métricas de entrenamiento son altas, lo que significa que el modelo capta bien los patrones del entrenamiento. En general, es el que mejor rendimiento obtuvo en test y entrenamiento, con métricas altas en precisión, recall y F1 Score.

Creemos que puede haber un cierto grado de sobreajuste porque hay mayor diferencia entre test y train, pero no es un overfitting demasiado elevado.

Y, finalmente, el set de entrenamiento del **modelo a elección (VotingClassifier)** tuvo la siguiente performance:

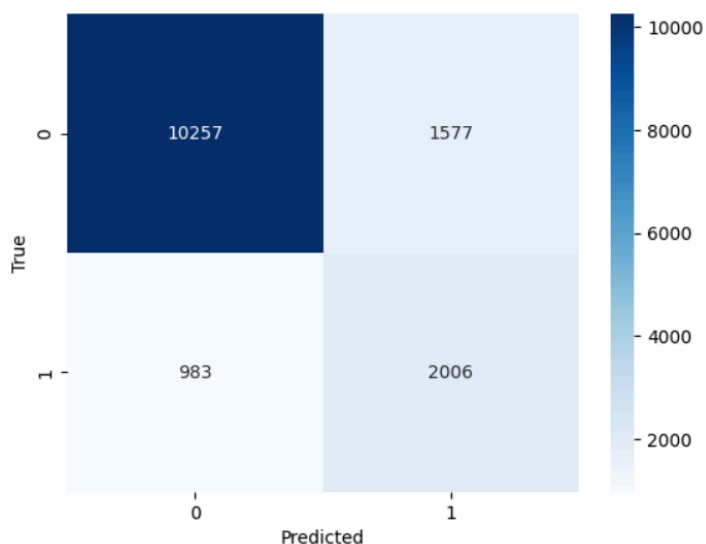
```
--- Entrenamiento ---  
Accuracy : 0.8718  
Precision: 0.6820  
Recall    : 0.6690  
F1 Score  : 0.6754
```

Observamos que el modelo es el más estable entre entrenamiento y test. Aunque sus métricas no fueron las más altas (superadas por el RandomForest), mantuvo un rendimiento constante con un F1 Score de 0.64 en test, muy cercano al de entrenamiento. Además, tiene equilibrio entre precisión y recall lo que lo hace una opción buena, aunque no se compara al rendimiento del Random Forest.

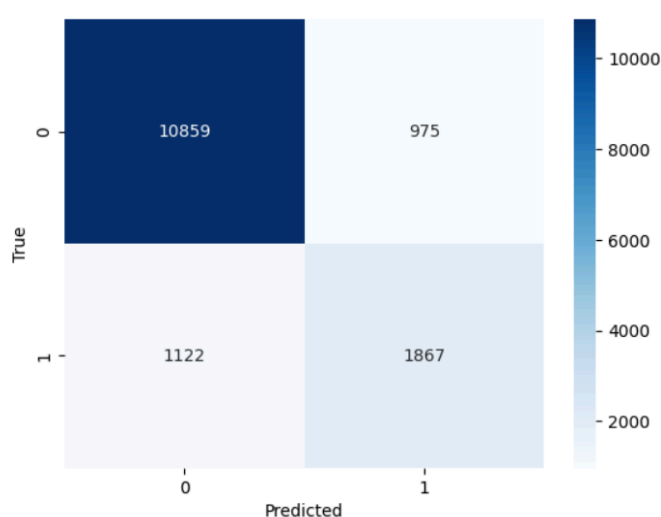
Para definir qué modelo elegiríamos si tuviéramos que predecir si lloverá o no al día siguiente, optamos por observar las matrices de confusión y la tabla de resultados.

Matrices de confusión:

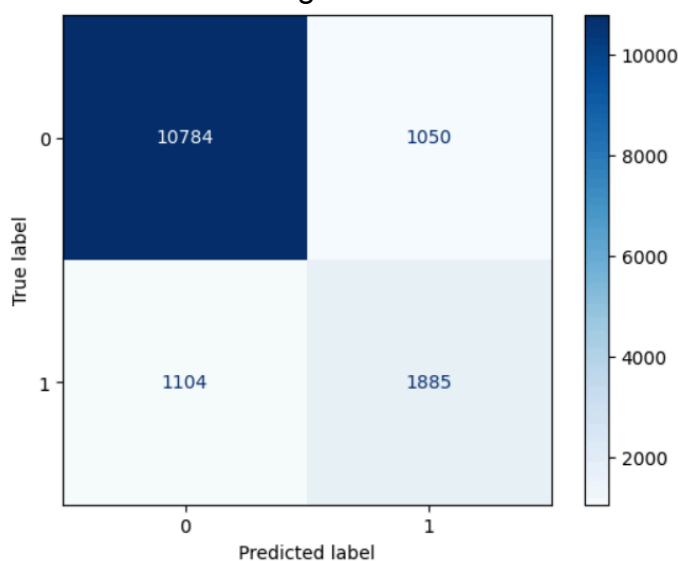
Árbol de Decisión



RandomForest



VotingClassifier



Fue una decisión difícil pero tras debatirlo concluimos que elegiríamos el **RandomForest** ya que es el que tiene mejores métricas a excepción del recall, que difiere por 0.006 con el VotingClassifier. Además, destacamos que el modelo alcanza el mejor F1 Score en el conjunto de prueba, lo que indica un equilibrio sólido entre precisión y recall, por lo que la diferencia con el VotingClassifier es aún menos significativa. Buscamos que estuviera balanceado, para que pudiera identificar correctamente los días en los que lloverá sin generar demasiadas falsas alarmas.

También su rendimiento en entrenamiento fue bueno, lo que muestra que tiene una alta capacidad predictiva. Teniendo en cuenta que aún hay un leve desbalance de clases, predice muy bien.

Analizando las tres matrices de confusión, la del Random Forest muestra que es el modelo que logra el mayor número de verdaderos negativos (10.859), lo que indica que es el más preciso al predecir que no lloverá. Solo comete 975 falsos positivos, que es el valor más bajo de los tres.

A pesar de que tiene un poco más de falsos negativos (1.122) que el Árbol de Decisión (983) o el VotingClassifier (1.104), mantiene un equilibrio razonable entre ambos tipos de error. Esto es clave porque, en un dataset desbalanceado como el nuestro, muchos modelos tienden a sobrepredecir la clase mayoritaria. Sin embargo, el Random Forest muestra ser capaz de identificar correctamente la clase minoritaria (RainTomorrow) con una tasa aceptable de falsos negativos, mientras que a la vez minimiza los falsos positivos, lo que lo convierte en una opción robusta si buscamos evitar alarmas innecesarias (que prediga lluvia cuando no la hay) sin perder demasiadas predicciones de lluvia reales.

Cuadro de tiempo dedicado

Integrante	Tarea	Prom. Hs Semana
Ramiro Barcala Roca	Implementación del Árbol de Decisión	6
Rafaela Pilar Arias	Implementación del RandomForest Implementación del Personalizado (VotingClassifier) Armado del Reporte	9
Mora Lema	Implementación del RandomForest Implementación del Personalizado (VotingClassifier) Armado del Reporte	9
Lara Eliana Ienco	Análisis inicial de variables	6